



Chapter 1. Installing Python

If you've picked up this book, you likely have Python installed on your machine already. Most common operating systems ship with a `python3` command. This can be the interpreter used by the system itself; on Windows and macOS, it's a placeholder that installs Python for you when you invoke it for the first time.

Why dedicate an entire chapter to the topic if it's so easy to get Python onto a new machine? The answer is that installing Python for long-term development can be a complex matter, and there are several reasons for this:

- You generally need multiple versions of Python installed side by side. (If you're wondering why, we'll get to that shortly.)
- There are a few different ways to install Python across the common platforms, each with unique advantages, tradeoffs, and sometimes pitfalls.
- Python is a moving target: you need to keep existing installations up-to-date with the latest maintenance release, add installations when a new feature version is published, and remove versions that are no longer supported. You may even need to test a prerelease of the next Python.
- You may want your code to run on multiple platforms. While Python makes it easy to write portable programs, setting up a developer environment requires some familiarity with the idiosyncrasies of each platform.
- You may want to run your code with an alternative implementation of Python.¹

In this first chapter, I'll show you how to install multiple Python versions on some of the major operating systems in a sustainable way—and how to keep your little snake farm in good shape.

TIP

Even if you develop for only a single platform, I'd encourage you to learn about working with Python on other operating systems. It's fun—and familiarity with other platforms enables you to provide a better experience to your software's contributors and users.

Supporting Multiple Versions of Python

Python programs often target several versions of the language and standard library at once. This may come as a surprise. Why would you run your code with anything but the latest Python? After all, this lets your programs benefit from new language features and library improvements immediately.

As it turns out, runtime environments often come with a variety of older versions of Python.² Even if you have tight control over your deployment environments, you may want to get into the habit of testing against multiple versions. The day the trusty Python in your production environment features in a security advisory had better not be the day you start porting your code to newer releases.

For these reasons, it's common to support both current and past versions of Python until their official end-of-life date and to set up installations for them side by side on a developer machine. With new feature versions coming out every year and support extending over five years, this gives you a testing matrix of five actively supported versions (see [Figure 1-1](#)). If that sounds like a lot of work, don't worry: the Python ecosystem comes with tooling that makes this a breeze.

Python has an annual release cycle: feature releases happen every October. Each feature release gets a new minor version in Python's `major.minor.micro` scheme. By contrast, new major versions are rare and reserved for strongly incompatible changes—as I write this in early 2024, a Python 4 is not in sight. Python's backward compatibility policy allows incompatible changes in minor releases when preceded by a two-year deprecation period.

Feature versions are maintained for five years, after which they reach end-of-life. Bugfix releases for a feature version occur roughly every other month during the first 18 months after its initial release.³ This is followed by security updates whenever necessary during the remainder of the five-year support period. Each maintenance release bumps the micro version.

Prereleases for upcoming Python feature releases happen throughout the year before their publication. These prereleases fall into three consecutive phases: alphas, betas, and release candidates. You can recognize them by the suffix that gets appended to the upcoming Python version, indicating the release status and sequence number, such as `a1`, `b3`, `rc2`.

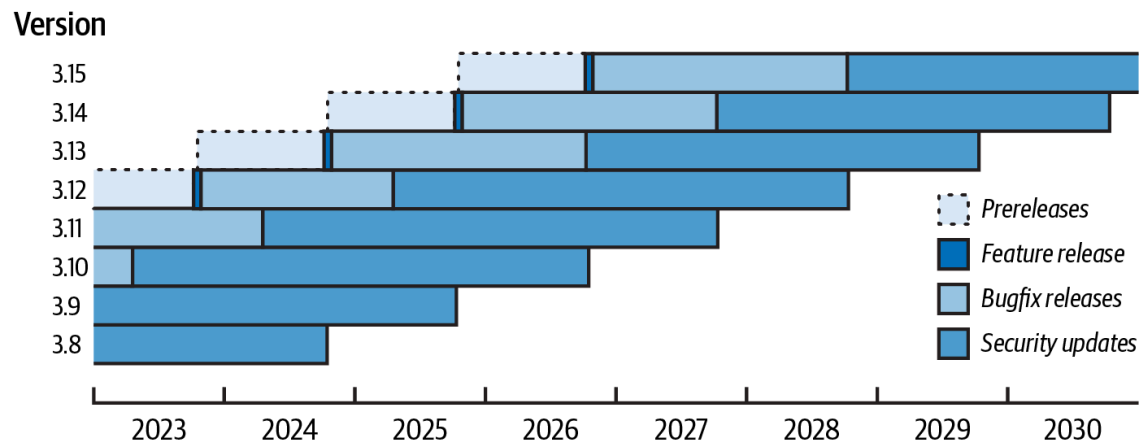


Figure 1-1. Timeline of Python releases

Locating Python Interpreters

How do you select the correct Python interpreter if you have multiple ones on your system? Let's look at a concrete example. When you type `python3` at the command line, the shell searches the directories in the `PATH` environment variable from left to right and invokes the first executable file named `python3`. Python installations on macOS and Linux also provide commands named `python3.12`, `python3.11`, and so on, to let you disambiguate between the different feature versions.

NOTE

On Windows, `PATH`-based interpreter discovery is less relevant because Python installations can be located via the Windows Registry (see [“The Python Launcher for Windows”](#)). Windows installers ship only an unversioned `python.exe` executable.

[Figure 1-2](#) shows a macOS machine with several Python installations. Starting from the bottom, the first interpreter is located in `/usr/bin/python3` and is part of Apple's Command Line Tools (Python 3.9 at time of writing). Next up, in `/opt/homebrew/bin`, are several interpreters from the Homebrew distribution; the `python3` command here is its main interpreter (Python 3.11). The Homebrew interpreters are followed by a prerelease from *python.org* (Python 3.13). The top entry contains the current release (Python 3.12 as of this writing), also from Homebrew.

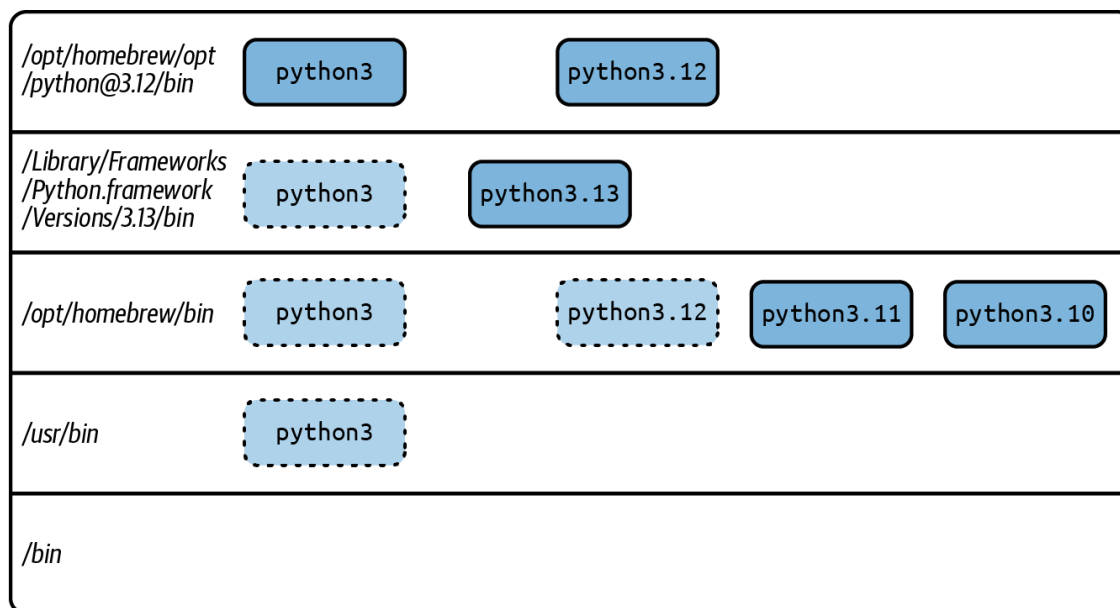


Figure 1-2. A developer system with multiple Python installations. The search path is displayed as a stack of directories; commands at the top shadow those further down.

The order of directories on the search path matters because earlier entries take precedence over, or “shadow,” later ones. In [Figure 1-2](#), `python3` refers to the current stable version (Python 3.12). If you omitted the top entry, `python3` would refer to the prerelease (Python 3.13). Without the top two entries, it would refer to Homebrew’s default interpreter, which is still on the previous stable version (Python 3.11).

Locating Python interpreters on `PATH` is a common source of errors. Some installations overwrite the `python3` command in shared directories such as `/usr/local/bin`. Others place `python3` in a distinct directory and alter the `PATH` to give it precedence, shadowing previously installed versions. To address these issues, this book uses the Python Launcher for Unix (see [“The Python Launcher for Unix”](#)). Nonetheless, understanding the `PATH` variable’s mechanics will help you avoid issues with Python discovery on Windows, macOS, and Linux.

A common default for the `PATH` variable is `/usr/local/bin:/usr/bin:/bin` on Unix-like systems, usually combined with some OS-dependent locations. You can modify the variable using the `export` built-in of many shells. Here’s how you would add a Python installation in `/usr/local/opt/python` using the Bash shell:

```
export PATH="/usr/local/opt/python/bin:$PATH"
```

You’re adding the *bin* subdirectory instead of the installation root because that’s where the interpreter is normally located on these systems. We’ll take a closer look at the layout of Python installations in [Chapter 2](#). Also, you’re adding the directory to the front of the `PATH` variable. I’ll explain shortly why this is usually what you want.

The previous line also works with Zsh, which is the default shell on macOS. That said, there’s a more idiomatic way to manipulate the search path on Zsh:

```
typeset -U path ❶
path=(/usr/local/opt/python/bin $path) ❷
```

- ❶ This instructs the shell to remove duplicate entries from the search path.
- ❷ The shell keeps the `path` array synchronized with the `PATH` variable.

The fish shell offers a function to uniquely and persistently prepend an entry to the search path:

```
fish_add_path /usr/local/opt/python/bin
```

It would be tedious to set up the search path manually at the start of every shell session. Instead, you can place the commands above in your *shell profile*—a file in your home directory that is read by the shell on startup. [Table 1-1](#) shows the most common ones.

Table 1-1. The startup files of some common shells

Shell	Startup file
Bash	<i>.bash_profile</i> (Debian and Ubuntu: <i>.profile</i>)
Zsh	<i>.zshrc</i>
fish	<i>.config/fish/fish.config</i>

Why is it important to add new directories to the front of the `PATH` variable? On a pristine macOS or Linux installation, the `python3` command often points to an old version of Python. As a Python developer, your default interpreter should be the latest stable release of Python. Prepending to `PATH` lets you control which Python installation your shell chooses when faced with an ambiguous command like `python3`. You can guarantee that `python3` points to the latest stable release of Python and that each `python3.x` points to the latest bugfix or security release of the 3.x line.

TIP

Unless your system already comes with a well-curated and up-to-date selection of interpreters, prepend Python installations to the `PATH` environment variable, with the latest stable version at the very front.

A SHORT HISTORY OF PATH

Curiously, the `PATH` mechanism has remained essentially the same since the 1970s. In the original design of the Unix operating system, the shell still looked up commands entered by the user in a directory named `/bin`. With the third edition of Unix (1973), this directory became too large for its 256K drive to hold all available programs. Researchers at Bell Labs introduced an additional filesystem hierarchy rooted at `/usr`, allowing a second disk to be mounted. But now the shell had to search for programs across multiple directories—`/bin` and `/usr/bin`. Eventually, the Unix designers settled on storing the list of directories in an environment variable named `PATH`. Since every process inherits its own copy of the environment, users can customize their search path without affecting system processes.

Installing Python on Windows

The core Python team provides official binary installers in the [Downloads for Windows section](#) of the Python website. Locate the latest release of

each Python version you wish to support, and download the 64-bit Windows installer for each.

NOTE

Depending on your domain and target environment, you may prefer to use the Windows Subsystem for Linux (WSL) for Python development. In this case, please refer to the section [“Installing Python on Linux”](#) instead.

In general, there should be little need to customize the installation—with one exception: when installing the latest stable release (and only then), enable the option to add Python to your `PATH` environment variable on the first page of the installer dialog. This ensures that your default `python` command uses a well-known and up-to-date Python version.

The *python.org* installers are an efficient way to set up multiversion Python environments on Windows, for these reasons:

- They register each Python installation with the Windows Registry, making it easy for developer tools to discover interpreters on the system (see [“The Python Launcher for Windows”](#)).
- They don’t have some of the disadvantages of redistributed versions of Python, such as lagging behind the official release or being subject to downstream modifications.
- They don’t require you to build the Python interpreter, which—apart from taking precious time—involves setting up Python’s build dependencies on your system.

Binary installers are provided only up to the last bugfix release of each Python version, which occurs around 18 months after the initial release. Security updates for older versions, on the other hand, are provided as source distributions only. If you don’t want to build Python from source,⁴ you can use one of the excellent [Python Standalone Builds](#), a collection of self-contained, highly portable Python distributions.

Keeping Python installations up-to-date falls on your shoulders when you’re using the binary installers from *python.org*. New releases are an-

nounced in many places, including the [Python blog](#) and the [Python Discourse](#). When you install a bugfix release for a Python version that is already present on the system, it will replace the existing installation. This preserves projects and developer tools on the upgraded Python version and should be a seamless experience.

When you install a new feature release of Python, be mindful of the following additional steps:

- Enable the option to add the new Python to the `PATH` environment variable.
- Remove the previous Python release from `PATH`. You can edit the environment variables for your account using the *System Settings* tool that is part of Windows.
- You may also wish to reinstall some of your developer tooling, to ensure that it runs on the latest Python version.

Eventually, a Python version will reach its end-of-life, and you may wish to uninstall it to free up resources. You can remove an existing installation using the *Installed Apps* tool. Choose the *Uninstall* action for its entry in the list of installed software. Beware that removing a Python version will break projects and tools that are still using it, so you should upgrade those to a newer Python beforehand.

MICROSOFT STORE PYTHON

Windows systems ship with a `python` stub that redirects the user to the latest Python package on the Microsoft Store. The Microsoft Store package is intended mainly for educational purposes and does not have full write access to some shared locations on the filesystem and registry. While it's useful for teaching Python to beginners, I would not recommend it for most intermediate and advanced Python development.

The Python Launcher for Windows

Python development on Windows is special in that tooling can locate Python installations via the Windows Registry. The Python Launcher for Windows leverages this to provide a single entry point to interpreters on the system. It is a utility included with every *python.org* release and associated with Python file extensions, allowing you to launch scripts from the Windows File Explorer.

Running applications with a double-click is handy, but the Python Launcher is at its most powerful when you invoke it from a command-line prompt. Open a PowerShell window and run the `py` command to start an interactive session:

```
> py
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb  6 2024, 21:26:36) [...] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

By default, the Python Launcher selects the most recent version of Python installed on the system. It's worth noting that this may not be the same as the *most recently installed* version on the system. This is good—you don't want your default Python to change when you install a bugfix release for an older version.

If you want to launch a specific version of the interpreter, you can pass the feature version as a command-line option:

```
> py -3.11
Python 3.11.8 (tags/v3.11.8:db85d51, Feb  6 2024, 22:03:32) [...] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Any remaining arguments to `py` are forwarded to the selected interpreter. Let's see how you would display the versions of two interpreters on the system:

```
> py -V
Python 3.12.2

> py -3.11 -V
Python 3.11.8
```

Using the same mechanism, you can run a script on a specific interpreter:

```
> py -3.11 path\to\script.py
```

NOTE

For historical reasons, `py` also inspects the first line of the script to see if a version is specified there. The canonical form is `#!/usr/bin/env python3`, which corresponds to `py -3` and works across all major platforms.

As you have seen, the Python Launcher defaults to the newest version on the system. There is an exception to this rule: if a *virtual environment* is active, `py` defaults to the interpreter in the virtual environment.⁵

When you install a prerelease of Python, the Python Launcher will use it as the default interpreter instead of the current release—after all, it’s the newest version on the system. In this case, you should override the default by setting the `PY_PYTHON` and `PY_PYTHON3` environment variables to the current release:

```
> setx PY_PYTHON 3.12
> setx PY_PYTHON3 3.12
```

Restart the console for the setting to take effect. Don’t forget to remove these variables once you upgrade from the prerelease to the final release.

To conclude our short tour of the Python Launcher, use the command `py --list` to enumerate the interpreters on your system:

```
> py --list
-V:3.13          Python 3.13 (64-bit)
-V:3.12 *        Python 3.12 (64-bit)
-V:3.11          Python 3.11 (64-bit)
-V:3.10          Python 3.10 (64-bit)
-V:3.9           Python 3.9 (64-bit)
-V:3.8           Python 3.8 (64-bit)
```

In this listing, the asterisk marks the default version of Python.

TIP

Even if you always use the Python Launcher yourself, you should still keep your `PATH` up-to-date. Some third-party tools run the `python.exe` command directly—you don't want them to use an outdated Python version or fall back to the Microsoft Store shim.

Installing Python on macOS

You can install Python on macOS in several ways. In this section, I'll take a look at the Homebrew package manager and the official *python.org* installers. Both provide multiversion binary distributions of Python. Some installation methods that are common on Linux—such as `pyenv`—also work on macOS. The Conda package manager even supports Windows, macOS, and Linux. I'll talk about them in later sections.

PYTHON FROM APPLE'S COMMAND LINE TOOLS

Apple's macOS ships with a `python3` stub that installs Apple's Command Line Tools when you run it for the first time, including an older version of Python. While it's good to know about Python commands on your system, other distributions will serve you better; they allow you to develop with Python versions of your choice.

Homebrew Python

Homebrew is a third-party package manager for macOS and Linux. It provides an *overlay distribution*, an open source software collection that you install on top of the existing operating system. Installing the package manager is straightforward; refer to the [official website](#) for instructions.

Homebrew distributes packages for every maintained feature version of Python. Use the `brew` command-line interface to manage them:

```
brew install python@3.x
```

Install a new Python version.

```
brew upgrade python@3.x
```

Upgrade a Python version to a maintenance release.

```
brew uninstall python@3.x
```

Uninstall a Python version.

NOTE

Whenever you see names like `python3.x` or `python@3.x` in this section, replace `3.x` with the actual feature version. For example, use `python3.12` and `python@3.12` for Python 3.12.

You may find that you already have some Python versions installed for other Homebrew packages that depend on them. Nonetheless, it's important that you install every version explicitly. Automatically installed packages may get deleted when you run `brew autoremove` to clean up resources.

Homebrew places a `python3.x` command for each version on your `PATH`, as well as a `python3` command for its main Python package—which may be either the current or the previous stable release. You should override this to ensure `python3` points to the latest version. First,

query the package manager for the installation root (which is platform-dependent):

```
$ brew --prefix python@3.12
/opt/homebrew/opt/python@3.12
```

Next, prepend the *bin* directory from this installation to your `PATH`. Here's an example that works on the Bash shell:

```
export PATH="/opt/homebrew/opt/python@3.12/bin:$PATH"
```

Homebrew has some advantages over the official *python.org* installers:

- You can use the command line to install, upgrade, and uninstall Python versions.
- Homebrew includes security releases for older versions—by contrast, *python.org* installers are provided up to the last bugfix release only.
- Homebrew Python is tightly integrated with the rest of the distribution. In particular, the package manager can satisfy Python dependencies like OpenSSL. This gives you the option to upgrade them independently when needed.

On the other hand, Homebrew Python also comes with some limitations:

- Homebrew doesn't package prereleases of upcoming Python versions.
- Packages generally lag a few days or weeks behind official releases. They also contain some downstream modifications, although these are quite reasonable. For example, Homebrew separates modules related to graphical user interfaces (GUI) from the main Python package.
- You can't install and uninstall Python packages system-wide unless they're also available as Homebrew packages. (See [“Virtual Environments”](#) for why you shouldn't install packages system-wide for development anyway.)
- Homebrew upgrades Python to maintenance releases automatically and in a way that can break virtual environments installed on the previous version.⁶

TIP

Personally, I recommend Hatch or Rye over Homebrew for managing Python on macOS (see [“A Brave New World: Installing with Hatch and Rye”](#)). Use the *python.org* installers to test your code against prereleases.

The python.org Installers

The core Python team provides official binary installers in the [downloads for macOS section of the Python website](#). Download the 64-bit *universal2* installer for the release you wish to install. The *universal2* binaries of the interpreter run natively on both Apple silicon and Intel chips.⁷

For multiversion development, I recommend a custom install—look for the *Customize* button in the installer dialog. In the resulting list of installable components, disable the *Unix command-line tools* and the *Shell profile updater*. Both options are designed to put the interpreter and some other commands on your `PATH`.⁸ Instead, edit your shell profile manually. Prepend the directory `/Library/Frameworks/Python.framework/Versions/3.x/bin` to `PATH`, replacing `3.x` with the actual feature version. Make sure the current stable release stays at the front of `PATH`.

NOTE

After installing a Python version, run the *Install Certificates* command located in the `/Applications/Python 3.x/` folder. This command installs Mozilla’s curated collection of root certificates, which are required to establish secure internet connections from Python.

When you install a bugfix release for a Python version that is already present on the system, it will replace the existing installation. You can uninstall a Python version by removing these two directories:

- `/Library/Frameworks/Python.framework/Versions/3.x/`
- `/Applications/Python 3.x/`

Most Python installations on Mac are so-called *framework builds*—a macOS concept for a “versioned bundle of shared resources.” You may have come across bundles before in the form of apps in the *Applications* folder. A *bundle* is just a directory with a standard layout, keeping all the files in one place.

Frameworks contain multiple versions side by side in directories named *Versions/3.x*. One of these is designated as the current version using a *Versions/Current* symbolic link (symlink). Under each version in a Python Framework, you can find a conventional Python installation layout with `bin` and `lib` directories.

Installing Python on Linux

The core Python team does not provide binary installers for Linux. Generally, the preferred way to install software on Linux distributions is using the official package manager. However, this isn’t unequivocally true when installing Python for development—here are some important caveats:

- The system Python in a Linux distribution may be quite old, and not all distributions include alternate Python versions in their main package repositories.
- Linux distributions have mandatory rules about how applications and libraries may be packaged. For example, Debian’s Python Policy mandates that the standard `ensurepip` module must be shipped in a separate package; as a result, you can’t create virtual environments on a default Debian system (a situation commonly fixed by installing the `python3-full` package).
- The main Python package in a Linux distribution serves as the foundation for other packages that require a Python interpreter. These packages may include critical parts of the system, such as Fedora’s package manager DNF. Distributions, therefore, apply safeguards to protect the

integrity of the system; for example, most distributions prevent you from installing or uninstalling packages system-wide using pip.

In the next sections, I'll take a look at installing Python on two major Linux distributions, Fedora and Ubuntu. Afterward, I'll cover some generic installation methods that don't use the official package manager: Homebrew, Nix, pyenv, and Conda. I'll also introduce you to the Python Launcher for Unix, a third-party package that aims to bring the `py` utility to Linux, macOS, and similar systems.

Fedora Linux

Fedora is an open source Linux distribution sponsored primarily by Red Hat and is the upstream source for Red Hat Enterprise Linux (RHEL). It aims to stay close to upstream projects and uses a rapid release cycle to foster innovation. Fedora is renowned for its excellent Python support, with Red Hat employing several Python core developers.

Python comes pre-installed on Fedora, and you can install additional Python versions using DNF:

```
sudo dnf install python3.x
```

Install a new Python version.

```
sudo dnf upgrade python3.x
```

Upgrade a Python version to a maintenance release.

```
sudo dnf remove python3.x
```

Uninstall a Python version.

Fedora has packages for all active feature versions and prereleases of CPython, the reference implementation of Python, as well as packages with alternative implementations like PyPy. A convenient shorthand to install all of these at once is to install the `tox` package:

```
$ sudo dnf install tox
```

In case you're wondering, tox is a test automation tool that makes it easy to run a test suite against multiple versions of Python; its Fedora package pulls in most available interpreters as recommended dependencies. Tox is also the spiritual ancestor of Nox, the subject of [Chapter 8](#).

Ubuntu Linux

Ubuntu is a popular Linux distribution based on Debian and funded by Canonical Ltd. Ubuntu ships only a single version of Python in its main repositories; other versions of Python, including prereleases, are provided by a *Personal Package Archive* (PPA). A PPA is a community-maintained software repository on Launchpad, the software collaboration platform run by Canonical.

Your first step on an Ubuntu system should be to add the `deadsnakes` PPA:

```
$ sudo apt update && sudo apt install software-properties-common
$ sudo add-apt-repository ppa:deadsnakes/ppa && sudo apt update
```

You can now install Python versions using the APT package manager:

```
sudo apt install python3.x-full
```

Install a new Python version.

```
sudo apt upgrade python3.x-full
```

Upgrade a Python version to a maintenance release.

```
sudo apt remove python3.x-full
```

Uninstall a Python version.

TIP

Always remember to include the `-full` suffix when installing Python on Debian and Ubuntu. The `python3.x-full` packages pull in the entire standard library and up-to-date root certificates. In particular, they ensure that you can create virtual environments.

Other Linux Distributions

What do you do if your Linux distribution doesn't package multiple versions of Python? The traditional answer is "roll your own Python." This may seem scary, but we'll see how straightforward building Python has become these days in ["Installing Python with pyenv"](#). However, it turns out that building from source is not your only option. Several cross-platform package managers provide binary packages of Python; in fact, you've already seen one of them.

The Homebrew distribution (see ["Homebrew Python"](#)) is available on Linux and macOS, and most of what I said above applies to Linux as well. The main difference between both platforms is the installation root: Homebrew on Linux installs packages under `/home/linuxbrew/.linuxbrew` by default instead of `/opt/homebrew`. Keep this in mind when adding Homebrew's Python installations to your `PATH`.

A popular cross-platform way to install Python is the Anaconda distribution, which is targeted at scientific computing and supports Windows, macOS, and Linux. I'll cover Anaconda in a separate section at the end of this chapter (see ["Installing Python from Anaconda"](#)).

Another fascinating option for both macOS and Linux is [Nix](#), a purely functional package manager with reproducible builds of thousands of software packages. Nix makes it fast and easy to set up isolated environments with arbitrary versions of software packages. Here's how you would set up a development environment with two Python versions:

```
$ nix-shell --packages python312 python311
[nix-shell]$ python3 -V
Python 3.12.1
[nix-shell]$ python3.11 -V
Python 3.11.7
[nix-shell]$ exit
```

Before dropping you into the environment, Nix transparently downloads prebuilt Python binaries from the Nix Packages collection and adds them to your `PATH`. Each package gets a unique subdirectory on the local filesystem, using a cryptographic hash that captures all its dependencies.

You can install the Nix package manager using its [official installer](#). If you're not ready to install Nix permanently, you can get a taste of what's possible using the Docker image for NixOS, a Linux distribution built entirely using Nix:

```
$ docker run --rm -it nixos/nix
```

The Python Launcher for Unix

The [Python Launcher for Unix](#) is a port of the official `py` utility to Linux and macOS, as well as any other operating system supporting the Rust programming language. Its key benefit is to offer a unified, cross-platform way to launch Python, with a well-defined default when no version is specified: the newest interpreter on the system.

The `py` command is a convenient, portable method for launching interpreters that avoids some pitfalls of invoking Python directly (see [“Locating Python Interpreters”](#)). For this reason, I’ll use it throughout this book. You can install the `python-launcher` package with a number of package managers, including Homebrew, DNF, and Cargo.

The Python Launcher for Unix discovers interpreters by scanning the `PATH` environment variable for `pythonx.y` commands. Otherwise, it works much like its Windows counterpart (see [“The Python Launcher for Windows”](#)). While `py` on its own launches the newest Python, you can also request a specific version—for example, `py -3.12` is equivalent to running `python3.12`.

The following is an example session using the macOS system from [Figure 1-2](#). (Python 3.13 was a prerelease at the time of writing this, so I’ve changed the default interpreter by setting `PY_PYTHON` and `PY_PYTHON3` to `3.12`.)

```
$ py -V
3.12.1
$ py -3.11 -V
3.11.7
$ py --list
3.13 | /Library/Frameworks/Python.framework/Versions/3.13/bin/python3.13
3.12 | /opt/homebrew/bin/python3.12
3.11 | /opt/homebrew/bin/python3.11
3.10 | /opt/homebrew/bin/python3.10
```

If a virtual environment is active, `py` defaults to the interpreter in that environment instead of the system-wide interpreter (see [“Virtual Environments”](#)). A special rule in the Python Launcher for Unix makes working with virtual environments more convenient: if the current directory (or one of its parents) contains a virtual environment with the standard name `.venv`, you don’t need to activate it explicitly.

You can run many third-party tools by passing their import name to the `-m` interpreter option. Suppose you have installed `pytest` (a test framework) on multiple Python versions. Using `py -m pytest` lets you deter-

mine which interpreter should run the tool. By contrast, a bare `pytest` uses the command that happens to appear first on your `PATH`.

If you invoke `py` with a Python script but don't specify a version, `py` inspects the first line of the script for a *shebang*—a line specifying the interpreter for the script. Stick with the canonical form here: `#!/usr/bin/env python3`. *Entry-point scripts* are a more sustainable way to link a script to a specific interpreter, because package installers can generate the correct interpreter path during installation (see [“Entry-point scripts”](#)).

WARNING

For compatibility with the Windows version, the Python Launcher uses only the Python version from shebangs, not the full interpreter path. As a result, you can end up with a different interpreter than if you were to invoke the script directly without `py`.

Installing Python with pyenv

Pyenv is a Python version manager for macOS and Linux. It includes a build tool—also available as a stand-alone program named `python-build`—that downloads, builds, and installs Python versions in your home directory. Pyenv allows you to activate and deactivate these installations globally, per project directory, or per shell session.

NOTE

In this section, you'll use pyenv as a build tool. If you're interested in using pyenv as a version manager, please refer to the [official documentation](#) for additional setup steps. I'll discuss some of the trade-offs in [“Managing Python Versions with pyenv”](#).

The best way to install pyenv on macOS and Linux is using Homebrew:

```
$ brew install pyenv sqlite3 xz zlib tcl-tk
```

Besides `pyenv` itself, this command also installs the build dependencies of Python. If you use a different installation method, check the [pyenv wiki](#) for platform-specific instructions on how to set up your build environment.

Display the available Python versions using the following command:

```
$ pyenv install --list
```

The list of interpreters is impressive. Not only does it cover all active feature versions of Python, but it also includes prereleases, unreleased development versions, almost every point release published over the past two decades, and a wealth of alternative implementations, such as PyPy, GraalPy, MicroPython, Jython, IronPython, and Stackless Python.

You can build and install any of these versions by passing them to `pyenv install`:

```
$ pyenv install 3.x.y
```

When using `pyenv` as a mere build tool, as we're doing here, you need to add each installation to `PATH` manually. You can find its location using the command `pyenv prefix 3.x.y` and append `/bin` to that. Here's an example for the Bash shell:

```
export PATH="$HOME/.pyenv/versions/3.x.y/bin:$PATH"
```

Installing a maintenance release with `pyenv` does *not* implicitly upgrade existing virtual environments and developer tools on the same feature version, so you'll have to recreate these environments using the new release.

When you no longer need an installation, you can remove it like this:

```
$ pyenv uninstall 3.x.y
```

By default, pyenv does not enable profile-guided optimization (PGO) or link-time optimization (LTO) when building the interpreter. According to the [Python Performance Benchmark Suite](#), these optimizations can lead to a significant speedup for CPU-bound Python programs—between 10% and 20%. You can enable them using the `PYTHON_CONFIGURE_OPTS` environment variable:

```
$ export PYTHON_CONFIGURE_OPTS='--enable-optimizations --with-lto'
```

Unlike most macOS installers, pyenv defaults to a POSIX installation layout instead of the framework builds typical for this platform. If you are on macOS, I advise you to enable framework builds for consistency.⁹ You can do so by adding the configuration option `--enable-framework` to the previous list.

Version management in pyenv works by placing small wrapper scripts called *shims* on your `PATH`. These shims intercept invocations of the Python interpreter and other Python-related tools and delegate to the actual commands in the appropriate Python installation. This can be seen as a more powerful method of interpreter discovery than the `PATH` mechanism of the operating system, and it avoids polluting the search path with a plethora of Python installations.

Pyenv's shim-based approach to version management is convenient, but it also comes with a tradeoff in runtime and complexity: shims add to the startup time of the Python interpreter. They also put deactivated commands on `PATH`, which can interfere with other tools that perform interpreter discovery, such as `py`, `virtualenv`, `tox`, and `Nox`. When installing packages with entry-point scripts, you need to run `pyenv rehash` for the scripts to become visible.

If the practical advantages of the shim mechanism convince you, you may also like [asdf](#), a generic version manager for multiple language runtimes; its Python plugin uses `python-build` internally. If you like per-directory version management but don't like shims, take a look at [direnv](#), which can update your `PATH` whenever you enter a directory. (It can even create and activate virtual environments for you.)

Installing Python from Anaconda

[Anaconda](#) is an open source software distribution for scientific computing, maintained by Anaconda Inc. Its centerpiece is [Conda](#), a cross-platform package manager for Windows, macOS, and Linux. Conda packages can contain software written in any language, such as C, C++, Python, R, or Fortran.

In this section, you'll use Conda to install Python. Conda does not install software packages globally on your system. Each Python installation is contained in a Conda environment and isolated from the rest of your sys-

tem. A typical Conda environment is centered around the dependencies of a particular project—say, a set of libraries for machine learning or data science—of which Python is only one among many.

Before you can create Conda environments, you'll need to bootstrap a base environment containing Conda itself. There are a few ways to go about this: you can install the full Anaconda distribution, or you can use the Miniconda installer with just Conda and a few core packages. Both Anaconda and Miniconda download packages from the *defaults* channel, which may require a commercial license for enterprise use.

Miniforge is a third alternative—it is similar to Miniconda but installs packages from the community-maintained *conda-forge* channel. You can get Miniforge using its official installers from [GitHub](#), or you can install it from Homebrew on macOS and Linux:

```
$ brew install miniforge
```

Conda requires shell integration to update the search path and shell prompt when you activate or deactivate an environment. If you've installed Miniforge from Homebrew, update your shell profile using the `conda init` command with the name of your shell. For example:

```
$ conda init bash
```

By default, the shell initialization code activates the base environment automatically in every session. You may want to disable this behavior if you also use Python installations that are not managed by Conda:

```
$ conda config --set auto_activate_base false
```

The Windows installer does not activate the base environment globally. Interact with Conda using the Miniforge Prompt from the Windows Start menu.

Congratulations, you now have a working Conda installation on your system! Let's use Conda to create an environment with a specific version of Python:

```
$ conda create --name=name python=3.x
```

Before you can use this Python installation, you need to activate the environment:

```
$ conda activate name
```

Upgrading Python to a newer release is simple:

```
$ conda update python
```

This command will run in the active Conda environment. What's great about Conda is that it won't upgrade Python to a release that's not yet supported by the Python libraries in the environment.

When you're done working in the environment, deactivate it like this:

```
$ conda deactivate
```

Conda doesn't install Python system-wide; rather, every Python installation is part of an isolated Conda environment. Conda takes a holistic view of an environment: Python is but one dependency of your projects, on par with system libraries, third-party Python packages, and even software packages from other language ecosystems.

A Brave New World: Installing with Hatch and Rye

While I was writing this book, the Python project managers [Rye](#) and [Hatch](#) added support for installing Python interpreters on all major plat-

forms. Both use interpreters from the [Python Standalone Builds collection](#) and the [PyPy project](#).

Both Rye and Hatch are distributed as stand-alone executables—in other words, you can easily install them on a system that doesn’t already have Python. Please refer to their official documentation for detailed installation instructions.

Hatch lets you install all CPython and PyPy interpreters compatible with your platform with a single command:

```
$ hatch python install all
```

This command also adds the installation directories to your `PATH`.¹⁰ Re-run the command with the `--update` option to upgrade the interpreters to newer versions. Hatch organizes interpreters by feature version, so patch releases overwrite the existing installation.

Rye fetches interpreters into the `~/.rye/py` directory. Normally, this happens behind the scenes when you synchronize the dependencies of your project. But it’s also available as a dedicated command:

```
$ rye fetch 3.12
$ rye fetch 3.11.8
$ rye fetch pypy@3.10
```

The second example places the interpreter in `~/.rye/py/cpython@3.11.8/bin` (Linux and macOS). You can install to another directory using the option `--target-path=<dir>`. This puts the interpreter in `<dir>` on Windows and `<dir>/bin` on Linux and macOS. Rye doesn’t add the interpreter to your `PATH` when you’re working outside of a project.

An Overview of Installers

[Figure 1-3](#) provides an overview of the main Python installation methods for Windows, macOS, and Linux.

Here's some case-by-case guidance on how to choose an installer:

- As a general rule, install Python stand-alone builds using Hatch.
- For scientific computing, I recommend using Conda instead.
- Get preleases from *python.org* if you're on Windows or macOS. If you're on Linux, build them from source with pyenv.
- On Fedora Linux, always use DNF.
- On Ubuntu Linux, always use the deadsnakes PPA with APT.

Choose Nix on macOS and Linux if you need a reproducible build of Python.

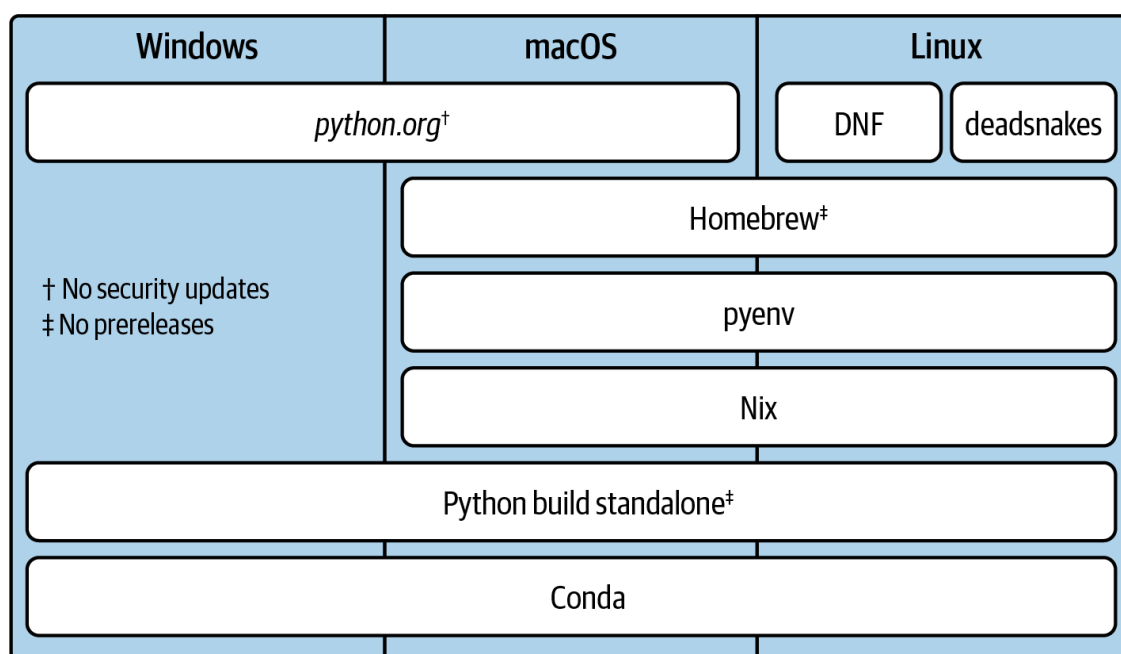


Figure 1-3. Python installers for Windows, macOS, and Linux

Summary

In this chapter, you've learned how to manage Python installations on Windows, macOS, and Linux. Use the Python Launcher to select interpreters installed on your system. Additionally, audit your search path to ensure you have well-defined `python` and `python3` commands.

The next chapter zooms into a Python installation: its contents and structure, and how your code interacts with it. You'll also learn about its lightweight cousins, virtual environments, and the tooling that has evolved around those.

- 1** While CPython is the reference implementation of Python, there are quite a few more to choose from: performance-oriented forks such as PyPy and Cinder, reimplementations such as RustPython and MicroPython, and ports to other platforms like WebAssembly, Java, and .NET.
- 2** At the time of writing in early 2024, the long-term support release of Debian Linux ships patched versions of Python 2.7.16 and 3.7.3—both released half a decade ago. (Debian’s “testing” distribution, which is widely used for development, comes with a current version of Python.)
- 3** Starting with Python 3.13, bugfix releases are provided for two years after the initial release.
- 4** [Stack Overflow](#) has a good step-by-step guide to building Windows installers.
- 5** [“Virtual Environments”](#) covers virtual environments in detail. For now, you can think of a virtual environment as a shallow copy of a full Python installation that lets you install a separate set of third-party packages.
- 6** Justin Mayer, [“Homebrew Python Is Not For You”](#), February 3, 2021.
- 7** Do you have a Mac with Apple silicon, but programs that must run on Intel processors? You’ll be pleased to know that the *python.org* installers also provide a `python3-intel64` binary using the `x86_64` instruction set. You can run it on Apple silicon thanks to Apple’s Rosetta translation environment.
- 8** The *Unix command-line tools* option places symbolic links in the `/usr/local/bin` directory, which can conflict with Homebrew packages and other versions from *python.org*. A *symbolic link* (symlink) is a special kind of file that points to another file, much like a shortcut in Windows.
- 9** For historical reasons, framework builds use a different path for the *per-user site directory*, the location where packages are installed if you invoke `pip` outside of a virtual environment and without administrative privileges. This different installation layout can prevent you from importing a previously installed package.
- 10** In a future release, Hatch will add interpreters to the Windows registry as well, letting you use them with the Python Launcher.

